

PHYS 4027 / 5027 • FALL 2026

Introduction to Computational Physics



Prof. Alex Chen

Department of Physics

Washington University in St. Louis

Logistics



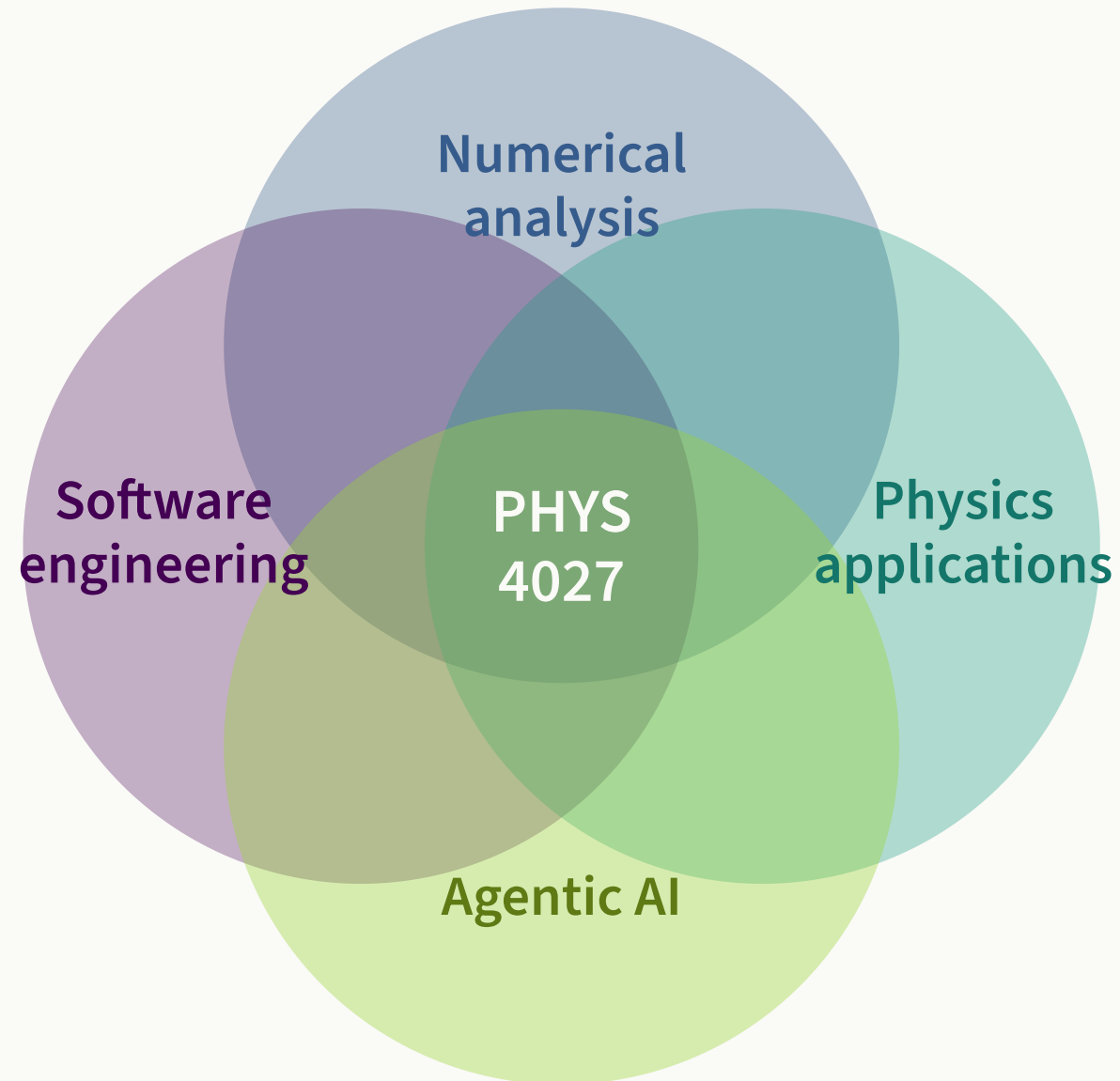
When & where

- Lectures: M/W/F 9:00–9:50am, Jubel 120
- Office hour: Tuesday 3:00–4:00pm, Compton 252
- Email: cyuran@wustl.edu

Materials

- [Course website](#) — all lecture slides
- [Canvas](#) — syllabus, announcements, quizzes
- [GitHub](#) — homework and projects

What is this course?



What this course is



- A course on using numerical algorithms to solve physics problems you cannot solve on paper.
- A course where you *implement* the algorithms — not just call a black-box library. We will not shy away from nitty-gritty implementation details.
- A course where you learn cutting-edge **agentic AI** and use it to get things done. AI use is not just permitted — the course is built assuming you use it for most of the coursework.
- A challenging course culminating in three projects involving complex physics.

What this course is not



- An algorithms course in the CS sense — no sorting, no AVL trees, no dynamic programming. Take a proper data structures course for that.
- A traditional physics course. We borrow examples from every branch of physics but cover none in depth; you either know it already, or you learn just enough.
- Career prep for software development. But what you learn here is very valuable for a career in quantitative science or engineering.

No prior programming experience is required — the first weeks bring everyone up to speed.

Why agentic AI?



- A modern AI agent can write a working ODE solver faster than you can read the assignment. It is the future of programming and computational physics in general.
- That does not make numerical methods obsolete. Implementation has never been the real bottleneck. It's at knowing what to do, verifying the implementation, and interpreting the physics.
- This semester: the agent may write your code — *you* own the predictions, the interpretation, and the diagnosis when it is wrong.
- To facilitate this, oral presentations are an integral part of the course.

The weekly rhythm



- **Wednesday:** homework goes out on GitHub Classroom.
- **Next Wednesday, 11:59pm:** homework due.
- **Friday of the same week:** short quiz on the homework material, then randomly drawn students present their solutions.

Everyone presents about **2 homework problems** over the semester — drawn without replacement, so no one is picked twice before everyone has gone once.

No late homework: Friday we discuss the solutions publicly — after that, the answers are in the room. Your slack is built-in instead: the lowest 2 homework scores and lowest 2 quiz scores are dropped automatically.

Projects



- **3 course projects**, in groups of 3, each with a written group report.
- Projects 1 and 2: topics and instructions given; group presentations in class, individually.
- Final project: **topic of your own choosing** — every group presents to the class.
- No midterm or final exam.

*Graduate students (5027): graduate-only groups, plus extra objectives on homework and projects.
Undergrads may attempt those for fun, not for points.*

Grading



Component	Weight
Homework submission	10%
Friday quizzes	10%
Homework presentations	10%
Project 1	20%
Project 2	20%
Final project	30%

Notice the shape: an agent can produce a homework artifact in minutes, so the handed-in files alone are worth little. The weight sits where understanding is demonstrated live — quizzes, presentations, and projects you will stand behind in person.

Tools of the trade



- **C++** — primary language; you implement the algorithms. [cppreference.com](https://en.cppreference.com) is the reference.
- **Python** — plotting and data analysis.
- **git + GitHub Classroom** — how homework and projects are distributed and collected.
- **VS Code** — our editor of choice (Vim/Emacs welcome).
- **Claude Code / Codex** — where much of the AI-driven coding work happens, via WashU's Claude Edu and OpenAI Edu access.

Action required this week

On Canvas, complete the assignment

“AI Literacy for the WashU Scholar”

Completing it grants you **Claude Edu** access — the AI platform we will use all semester.

References



- *Numerical Recipes: The Art of Scientific Computing*, 3rd ed. (2007) — covers most of the course in depth; free to read at the [official website](#).
- *Computational Physics*, Mark Newman.
- *Algorithms in C++*, Robert Sedgewick.

No required textbook — the slides and homework are self-contained; these are for depth.

Floating point numbers

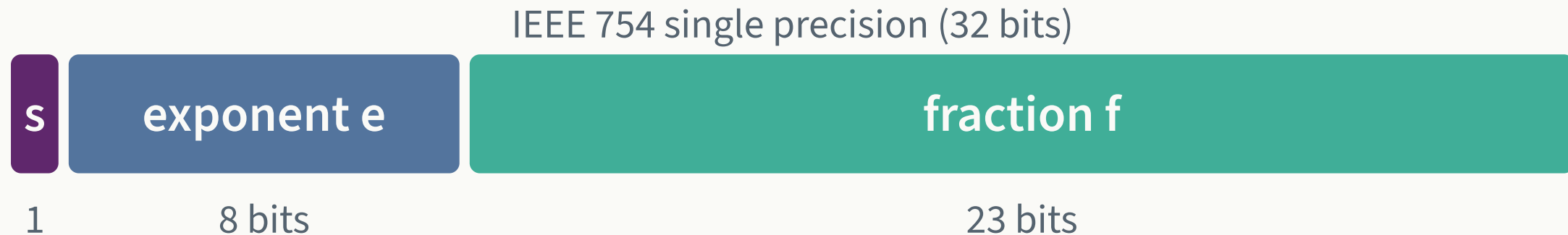


How computers represent the real line and where it breaks

A finite budget of bits

There are uncountably many real numbers; a `float` has 32 bits — at most 2^{32} distinct values to cover them all.

The solution is binary scientific notation: store a sign, a fraction, and an exponent. The "fraction" is also called "significand" or "mantissa".



$$x = (-1)^s \times 1.f \times 2^{e-127}$$

The C++ floating point types



Type	Bits	~Decimal digits	Typical use
<code>float</code>	32	7	graphics, games, ML
<code>double</code>	64	16	scientific computing's workhorse
<code>long double</code>	80/128	≥ 18	occasional extra headroom

Machine learning pushed the other way: `fp16`, `bf16`, `fp8`, even `fp4` — trading accuracy for speed and memory.

Not every decimal exists in binary

0.1 has no finite binary representation — just as $1/3$ has none in decimal. The computer stores the nearest representable number instead.

Code

```
double a = (1.0 - 0.1) - 0.9;  
double b = (1.0 - 0.05 - 0.05) - 0.9;  
std::cout << a << std::endl;  
std::cout << b << std::endl;
```

Output

```
0  
-1.11022e-16
```

Rounding makes arithmetic non-associative: $(a + b) + c$ need not equal $a + (b + c)$.

Machine epsilon and ULP



Unit in the Last Place (ULP): the distance between two adjacent floating point numbers — the change from flipping the last bit of the fraction.

The ULP scales with the exponent, so we usually quote the ULP at exponent 0. That number is the **machine epsilon**: the relative precision of the format.

1 ULP is the best accuracy any single floating point operation can promise. Expecting more from a calculation is expecting magic.

Catastrophic cancellation

Subtracting two nearly equal numbers cancels their leading digits, leaving only the trailing bits — which are rounding noise.

Code

```
double x = 1.0e-8;
double naive = 1.0 - std::cos(x);
double stable =
    2.0 * std::sin(0.5 * x)
        * std::sin(0.5 * x);
cout << "naive:" << naive << endl;
cout << "stable:" << stable << endl;
```

Output

```
naive: 0
stable: 5e-17
```

True answer: $1 - \cos x \approx x^2/2 = 5 \times 10^{-17}$.
The naive version is 100% wrong.

The edges of the format

- **Two zeros:** $+0$ and -0 exist, and compare equal.
- **Infinities:** `1.0/0.0` gives `inf` — also produced by overflow.
- **NaN** (“not a number”): `0.0/0.0`, `sqrt(-1.0)` ... It propagates through arithmetic and is the only value with `x  $\neq$  x`.
- **Subnormals:** when the exponent is zero, the number convention becomes slightly different, decreasing uniformly to 0.

Never compare floats directly



Test whether the difference is within a tolerance instead:

Do NOT

```
double a = 1.345;
double b = 1.123;
double c = a + b;
if (c == 2.468) { ... }
```

Do

```
double a = 1.345;
double b = 1.123;
double c = a + b;
double EPSILON = 1e-10;
if (std::abs(c - 2.468) < EPSILON) {
    ...
}
```

Choosing the tolerance is problem-dependent. We will return to this all semester.

One more ambush: integer arithmetic

- Write `1.0` and C++ reads a `double`; write `1` and it reads an integer.
- Operations between integers give integers: `1/2` is `0`, not `0.5`.
- Always write numbers with a decimal point and this class of bug never happens to you.

Where this goes next



- Homework 1 (out Wednesday) is a floating point laboratory: you will probe the limits of each type experimentally, and reconcile what you predicted with what you measure.
- Deeper reading: [What every computer scientist should know about floating-point arithmetic](#) (Goldberg 1991).
- **Next lecture:** a crash course in C++.

Before you leave: bookmark the course website, and start the “**AI Literacy for the WashU Scholar**” assignment on Canvas tonight.